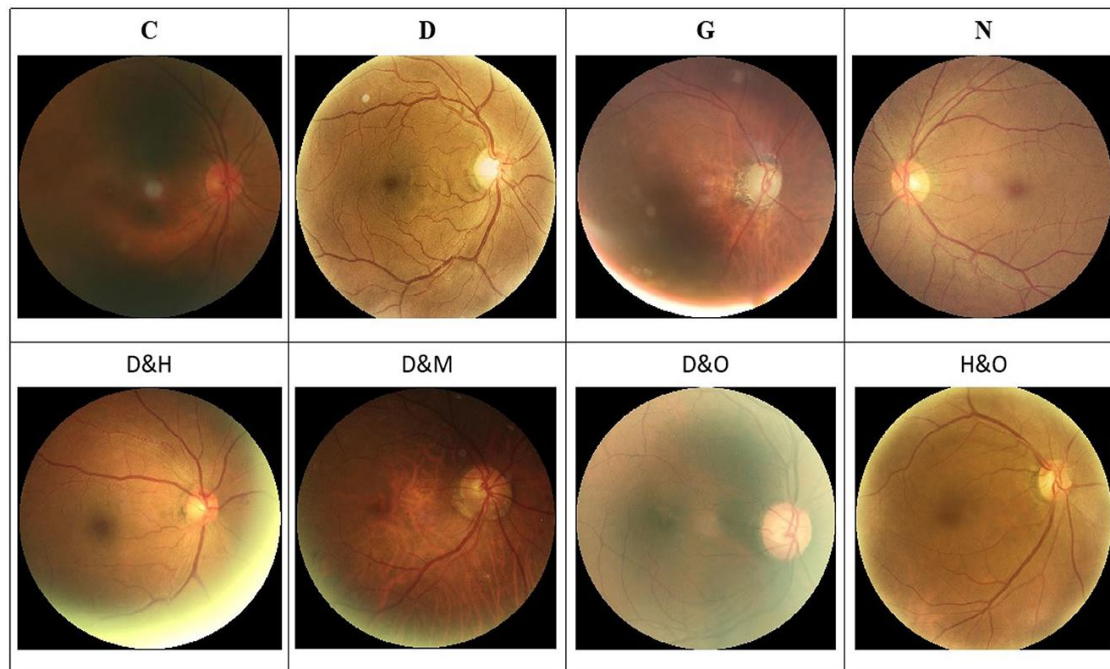


Retinal Disease Classification using VGG16, InceptionV3 and DenseNet121 with NIRMAL Optimizer



```
import numpy as np
import pandas as pd
import os

image_dir = '/kaggle/input/retinal-disease-detection/Diabetic
Retinopathy/train/images'
csv_path = '/kaggle/input/retinal-disease-detection/Diabetic
Retinopathy/train/train_annotations.csv'

df = pd.read_csv(csv_path)

df['image_path'] = df['Image name'].apply(lambda x: os.path.join(image_dir,
x))

df = df.rename(columns={
    'Image name': 'image_name',
    'Retinopathy grade': 'retinopathy_grade',
    'Risk of macular edema': 'risk_of_macular_edema'
})

df = df[['image_path', 'retinopathy_grade', 'risk_of_macular_edema']]

df
```

	image_path	retinopathy_grade	\
0	/kaggle/input/retinal-disease-detection/Diabet...	2	
1	/kaggle/input/retinal-disease-detection/Diabet...	0	
2	/kaggle/input/retinal-disease-detection/Diabet...	0	
3	/kaggle/input/retinal-disease-detection/Diabet...	1	
4	/kaggle/input/retinal-disease-detection/Diabet...	1	
...	...	...	
1572	/kaggle/input/retinal-disease-detection/Diabet...	0	
1573	/kaggle/input/retinal-disease-detection/Diabet...	1	
1574	/kaggle/input/retinal-disease-detection/Diabet...	0	
1575	/kaggle/input/retinal-disease-detection/Diabet...	2	
1576	/kaggle/input/retinal-disease-detection/Diabet...	0	

	risk_of_macular_edema
0	0
1	0
2	0
3	0
4	0
...	...
1572	0
1573	0
1574	0
1575	0
1576	0

[1577 rows x 3 columns]

df.columns

Index(['image_path', 'retinopathy_grade', 'risk_of_macular_edema'],
dtype='object')

df.shape

(1577, 3)

df.duplicated().sum()

0

df.isnull().sum()

image_path	0
retinopathy_grade	0
risk_of_macular_edema	0
dtype: int64	

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1577 entries, 0 to 1576

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	image_path	1577 non-null	object
1	retinopathy_grade	1577 non-null	int64
2	risk_of_macular_edema	1577 non-null	int64

dtypes: int64(2), object(1)

memory usage: 37.1+ KB

```
df['retinopathy_grade'].unique()
```

```
array([2, 0, 1, 4, 3])
```

```
df['retinopathy_grade'].value_counts()
```

```
retinopathy_grade
```

```
0    829
```

```
2    358
```

```
1    206
```

```
3    116
```

```
4     68
```

```
Name: count, dtype: int64
```

```
df['risk_of_macular_edema'].unique()
```

```
array([0, 1])
```

```
df['risk_of_macular_edema'].value_counts()
```

```
risk_of_macular_edema
```

```
0    1268
```

```
1     309
```

```
Name: count, dtype: int64
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
sns.set_style("whitegrid")
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
sns.countplot(data=df, x='retinopathy_grade')
```

```
plt.title('Distribution of Retinopathy Grade')
```

```
plt.xlabel('Retinopathy Grade')
```

```
plt.ylabel('Count')
```

```
plt.subplot(1, 2, 2)
```

```
sns.countplot(data=df, x='risk_of_macular_edema')
```

```
plt.title('Distribution of Risk of Macular Edema')
```

```
plt.xlabel('Risk of Macular Edema')
```

```
plt.ylabel('Count')
```

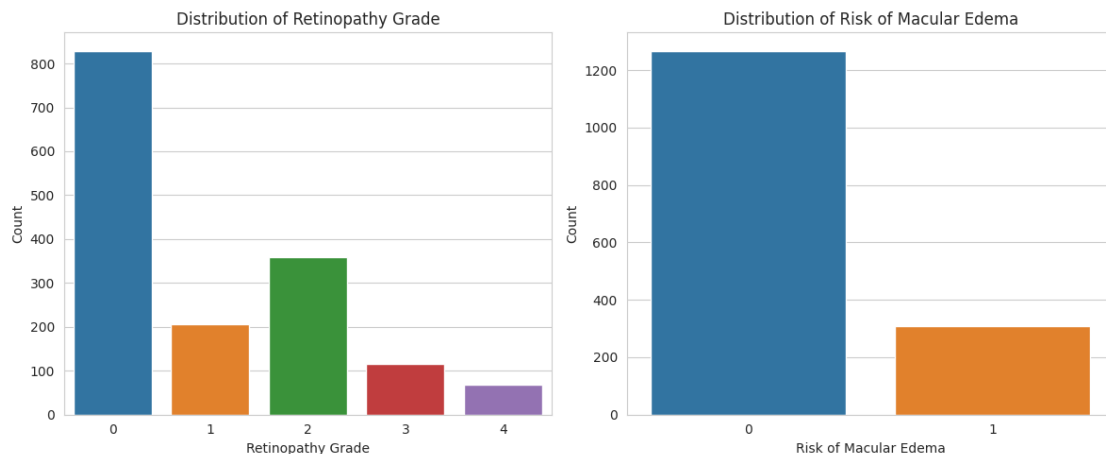
```
plt.tight_layout()  
plt.show()
```

```
plt.figure(figsize=(12, 5))
```

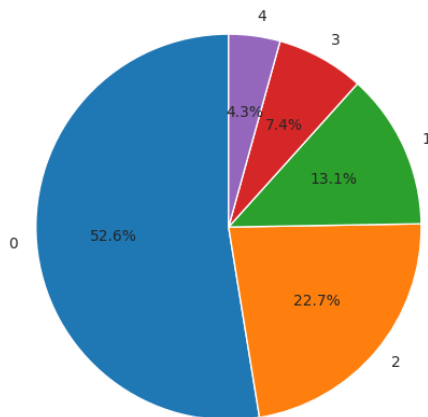
```
plt.subplot(1, 2, 1)  
retinopathy_counts = df['retinopathy_grade'].value_counts()  
plt.pie(retinopathy_counts, labels=retinopathy_counts.index,  
autopct='%1.1f%%', startangle=90)  
plt.title('Proportion of Retinopathy Grade')
```

```
plt.subplot(1, 2, 2)  
edema_counts = df['risk_of_macular_edema'].value_counts()  
plt.pie(edema_counts, labels=edema_counts.index, autopct='%1.1f%%',  
startangle=90)  
plt.title('Proportion of Risk of Macular Edema')
```

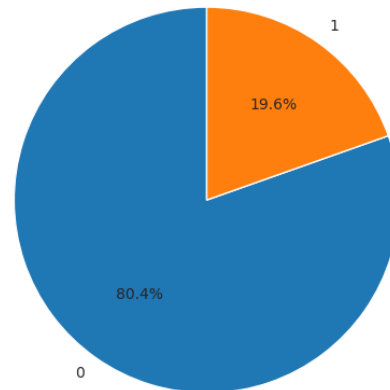
```
plt.tight_layout()  
plt.show()
```



Proportion of Retinopathy Grade



Proportion of Risk of Macular Edema



```
from PIL import Image
```

```
def display_images_by_category(df, column_name, num_images=5):
    categories = df[column_name].unique()
```

```
    for category in categories:
        category_df = df[df[column_name] == category]
```

```
        category_images = category_df['image_path'].head(num_images).tolist()
```

```
        plt.figure(figsize=(15, 3))
        plt.suptitle(f'{column_name}: {category}', fontsize=16)
```

```
        for i, image_path in enumerate(category_images):
```

```
            if os.path.exists(image_path):
                img = Image.open(image_path)
                plt.subplot(1, num_images, i + 1)
                plt.imshow(img)
                plt.axis('off')
                plt.title(f'Image {i+1}')
```

```
            else:
                print(f"Image not found: {image_path}")
```

```
        plt.tight_layout()
        plt.show()
```

```
display_images_by_category(df, 'retinopathy_grade', num_images=5)
```

```
display_images_by_category(df, 'risk_of_macular_edema', num_images=5)
```

retinopathy_grade: 2



retinopathy_grade: 0



retinopathy_grade: 1



retinopathy_grade: 4



retinopathy_grade: 3



risk_of_macular_edema: 0



risk_of_macular_edema: 1



```
from sklearn.utils import resample
```

```
max_count = df['risk_of_macular_edema'].value_counts().max()
```

```
dfs = []
for category in df['risk_of_macular_edema'].unique():
    class_subset = df[df['risk_of_macular_edema'] == category]
    class_upsampled = resample(class_subset,
                               replace=True,
                               n_samples=max_count,
                               random_state=42)
    dfs.append(class_upsampled)
```

```
df_balanced = pd.concat(dfs).sample(frac=1,
                                     random_state=42).reset_index(drop=True)
```

```
df = df_balanced
```

```
df
```

	image_path	retinopathy_grade \
0	/kaggle/input/retinal-disease-detection/Diabet...	3
1	/kaggle/input/retinal-disease-detection/Diabet...	3
2	/kaggle/input/retinal-disease-detection/Diabet...	0
3	/kaggle/input/retinal-disease-detection/Diabet...	3
4	/kaggle/input/retinal-disease-detection/Diabet...	2
...	...	...
2531	/kaggle/input/retinal-disease-detection/Diabet...	3
2532	/kaggle/input/retinal-disease-detection/Diabet...	0
2533	/kaggle/input/retinal-disease-detection/Diabet...	0
2534	/kaggle/input/retinal-disease-detection/Diabet...	4
2535	/kaggle/input/retinal-disease-detection/Diabet...	0

	risk_of_macular_edema
0	1
1	0
2	0
3	1
4	1
...	...
2531	1
2532	0
2533	0
2534	1
2535	0

```
[2536 rows x 3 columns]
```

```
df = df[['image_path', 'risk_of_macular_edema']]
```

```
df
```

	image_path
risk_of_macular_edema	

```

0      /kaggle/input/retinal-disease-detection/Diabet...
1
1      /kaggle/input/retinal-disease-detection/Diabet...
0
2      /kaggle/input/retinal-disease-detection/Diabet...
0
3      /kaggle/input/retinal-disease-detection/Diabet...
1
4      /kaggle/input/retinal-disease-detection/Diabet...
1
...
...
2531  /kaggle/input/retinal-disease-detection/Diabet...
1
2532  /kaggle/input/retinal-disease-detection/Diabet...
0
2533  /kaggle/input/retinal-disease-detection/Diabet...
0
2534  /kaggle/input/retinal-disease-detection/Diabet...
1
2535  /kaggle/input/retinal-disease-detection/Diabet...
0

```

[2536 rows x 2 columns]

```

import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from PIL import Image
from torch.optim import Optimizer

class NIRMAL(Optimizer):
    def __init__(self, params, lr=1e-3, momentum=0.9, beta=0.999, eps=1e-8,
                 knight_scale=0.01, camel_scale=1.5, horse_scale=0.5,
                 w_wazir=0.3, w_elephant=0.25, w_knight=0.1, w_camel=0.2,
                 w_horse=0.15, weight_decay=0):
        defaults = dict(
            lr=lr, momentum=momentum, beta=beta, eps=eps,
            knight_scale=knight_scale, camel_scale=camel_scale,
            horse_scale=horse_scale,
            w_wazir=w_wazir, w_elephant=w_elephant, w_knight=w_knight,
            w_camel=w_camel, w_horse=w_horse, weight_decay=weight_decay
        )

```



```

    super(NIRMAL, self).__init__(params, defaults)

def __setstate__(self, state):
    super(NIRMAL, self).__setstate__(state)

def step(self, closure=None):
    loss = None
    if closure is not None:
        loss = closure()

    for group in self.param_groups:
        lr = group['lr']
        momentum = group['momentum']
        beta = group['beta']
        eps = group['eps']
        knight_scale = group['knight_scale']
        camel_scale = group['camel_scale']
        horse_scale = group['horse_scale']
        w_wazir = group['w_wazir']
        w_elephant = group['w_elephant']
        w_knight = group['w_knight']
        w_camel = group['w_camel']
        w_horse = group['w_horse']
        weight_decay = group['weight_decay']

        for p in group['params']:
            if p.grad is None:
                continue
            grad = p.grad.data

            if weight_decay != 0:
                grad = grad.add(p.data, alpha=weight_decay)

            state = self.state[p]

            if len(state) == 0:
                state['step'] = 0
                state['m'] = torch.zeros_like(p.data)
                state['v'] = torch.zeros_like(p.data)

            m, v = state['m'], state['v']
            state['step'] += 1

            delta_wazir = -lr * grad
            m.mul_(momentum).add_(grad, alpha=1 - momentum)
            delta_elephant = -lr * m
            delta_knight = lr * knight_scale * torch.randn_like(p.data)
            v.mul_(beta).addcmul_(grad, grad, value=1 - beta)
            delta_camel = -lr * camel_scale * m / (torch.sqrt(v) + eps)

```

```

        delta_horse = -lr * horse_scale * torch.tanh(m)

        delta_total = (w_wazir * delta_wazir + w_elephant *
delta_elephant +
                        w_knight * delta_knight + w_camel * delta_camel
+
                        w_horse * delta_horse)

        p.data.add_(delta_total)

    return loss

```

```

class RetinalDataset(Dataset):

```

```

    def __init__(self, dataframe, transform=None):
        self.dataframe = dataframe
        self.transform = transform

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, idx):
        img_path = self.dataframe.iloc[idx]['image_path']
        image = Image.open(img_path).convert('RGB')
        label = self.dataframe.iloc[idx]['risk_of_macular_edema']
        if self.transform:
            image = self.transform(image)
        return image, label

```

```

transform_224 = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

```

```

transform_299 = transforms.Compose([
    transforms.Resize((299, 299)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

```

```

train_size = int(0.8 * len(df))
val_size = len(df) - train_size
train_df, val_df = torch.utils.data.random_split(df, [train_size, val_size])

```

```

train_dataset_224 = RetinalDataset(train_df.dataset, transform=transform_224)
val_dataset_224 = RetinalDataset(val_df.dataset, transform=transform_224)
train_dataset_299 = RetinalDataset(train_df.dataset, transform=transform_299)
val_dataset_299 = RetinalDataset(val_df.dataset, transform=transform_299)

```

```

train_loader_224 = DataLoader(train_dataset_224, batch_size=32, shuffle=True)
val_loader_224 = DataLoader(val_dataset_224, batch_size=32, shuffle=False)
train_loader_299 = DataLoader(train_dataset_299, batch_size=32, shuffle=True)
val_loader_299 = DataLoader(val_dataset_299, batch_size=32, shuffle=False)

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

def get_model(model_name):
    if model_name == "vgg16":
        model = models.vgg16(weights='IMAGENET1K_V1')
        model.classifier[6] = nn.Linear(4096, 2)
    elif model_name == "inception":
        model = models.inception_v3(weights='IMAGENET1K_V1')
        model.fc = nn.Linear(2048, 2)
    elif model_name == "densenet121":
        model = models.densenet121(weights='IMAGENET1K_V1')
        model.classifier = nn.Linear(1024, 2)
    return model.to(device)

```

```

models_list = ["vgg16", "inception", "densenet121"]
history = {model: {"train_loss": [], "val_loss": [], "train_acc": [],
"val_acc": []} for model in models_list}
criterion = nn.CrossEntropyLoss()

```

```

for model_name in models_list:
    model = get_model(model_name)
    optimizer = NIRMAL(model.parameters())
    train_loader = train_loader_299 if model_name == "inception" else
train_loader_224
    val_loader = val_loader_299 if model_name == "inception" else
val_loader_224
    for epoch in range(5):
        model.train()
        train_loss, train_correct, train_total = 0.0, 0, 0
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            if model_name == "inception":
                outputs, aux_outputs = model(inputs)
                loss1 = criterion(outputs, labels)
                loss2 = criterion(aux_outputs, labels)
                loss = loss1 + 0.4 * loss2
            else:
                outputs = model(inputs)
                loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            train_loss += loss.item()
            _, predicted = torch.max(outputs, 1)

```

```

        train_total += labels.size(0)
        train_correct += (predicted == labels).sum().item()
    train_loss /= len(train_loader)
    train_acc = train_correct / train_total
    model.eval()
    val_loss, val_correct, val_total = 0.0, 0, 0
    all_preds, all_labels = [], []
    with torch.no_grad():
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
    val_loss /= len(val_loader)
    val_acc = val_correct / val_total
    history[model_name]["train_loss"].append(train_loss)
    history[model_name]["val_loss"].append(val_loss)
    history[model_name]["train_acc"].append(train_acc)
    history[model_name]["val_acc"].append(val_acc)
    cm = confusion_matrix(all_labels, all_preds)
    plt.figure(figsize=(6, 6))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title(f"Confusion Matrix - {model_name}")
    plt.ylabel("True Label")
    plt.xlabel("Predicted Label")
    plt.savefig(f"cm_{model_name}.png")
    plt.close()
    print(f"Classification Report - {model_name}:")
    print(classification_report(all_labels, all_preds, target_names=["No
Risk", "Risk"]))

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
for model_name in models_list:
    plt.plot(history[model_name]["train_loss"], label=f"{model_name} Train")
    plt.plot(history[model_name]["val_loss"], label=f"{model_name} Val")
plt.title("Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.subplot(1, 2, 2)
for model_name in models_list:
    plt.plot(history[model_name]["train_acc"], label=f"{model_name} Train")
    plt.plot(history[model_name]["val_acc"], label=f"{model_name} Val")
plt.title("Accuracy Comparison")

```

```
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.tight_layout()
plt.savefig("comparison_plot.png")
plt.close()
```

Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to /root/.cache/torch/hub/checkpoints/vgg16-397923af.pth
 100%|██████████| 528M/528M [00:02<00:00, 213MB/s]

Classification Report - vgg16:

	precision	recall	f1-score	support
No Risk	0.78	0.73	0.75	1268
Risk	0.75	0.79	0.77	1268
accuracy			0.76	2536
macro avg	0.76	0.76	0.76	2536
weighted avg	0.76	0.76	0.76	2536

Downloading: "https://download.pytorch.org/models/inception_v3_google-0cc3c7bd.pth" to /root/.cache/torch/hub/checkpoints/inception_v3_google-0cc3c7bd.pth
 100%|██████████| 104M/104M [00:00<00:00, 211MB/s]

Classification Report - inception:

	precision	recall	f1-score	support
No Risk	1.00	1.00	1.00	1268
Risk	1.00	1.00	1.00	1268
accuracy			1.00	2536
macro avg	1.00	1.00	1.00	2536
weighted avg	1.00	1.00	1.00	2536

Downloading: "https://download.pytorch.org/models/densenet121-a639ec97.pth" to /root/.cache/torch/hub/checkpoints/densenet121-a639ec97.pth
 100%|██████████| 30.8M/30.8M [00:00<00:00, 159MB/s]

Classification Report - densenet121:

	precision	recall	f1-score	support
No Risk	0.97	0.98	0.97	1268
Risk	0.98	0.97	0.97	1268
accuracy			0.97	2536
macro avg	0.97	0.97	0.97	2536

weighted avg	0.97	0.97	0.97	2536
--------------	------	------	------	------

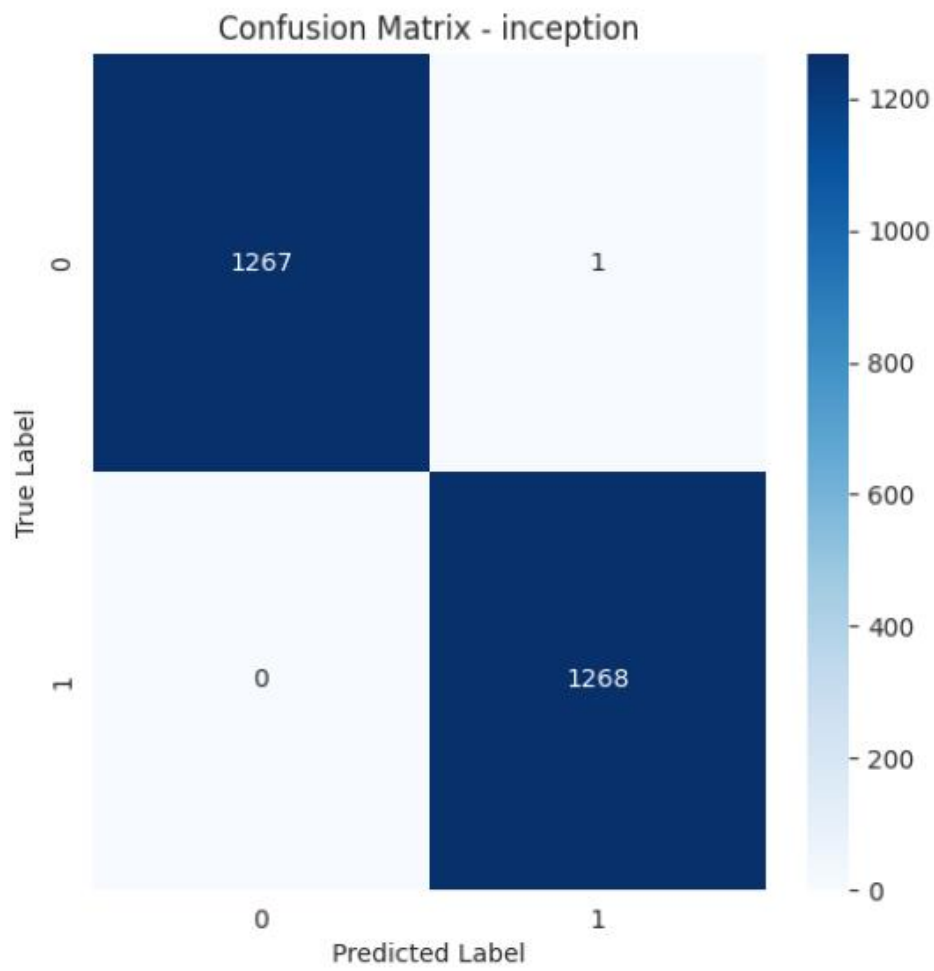
```
import os
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

path = "/kaggle/working/"

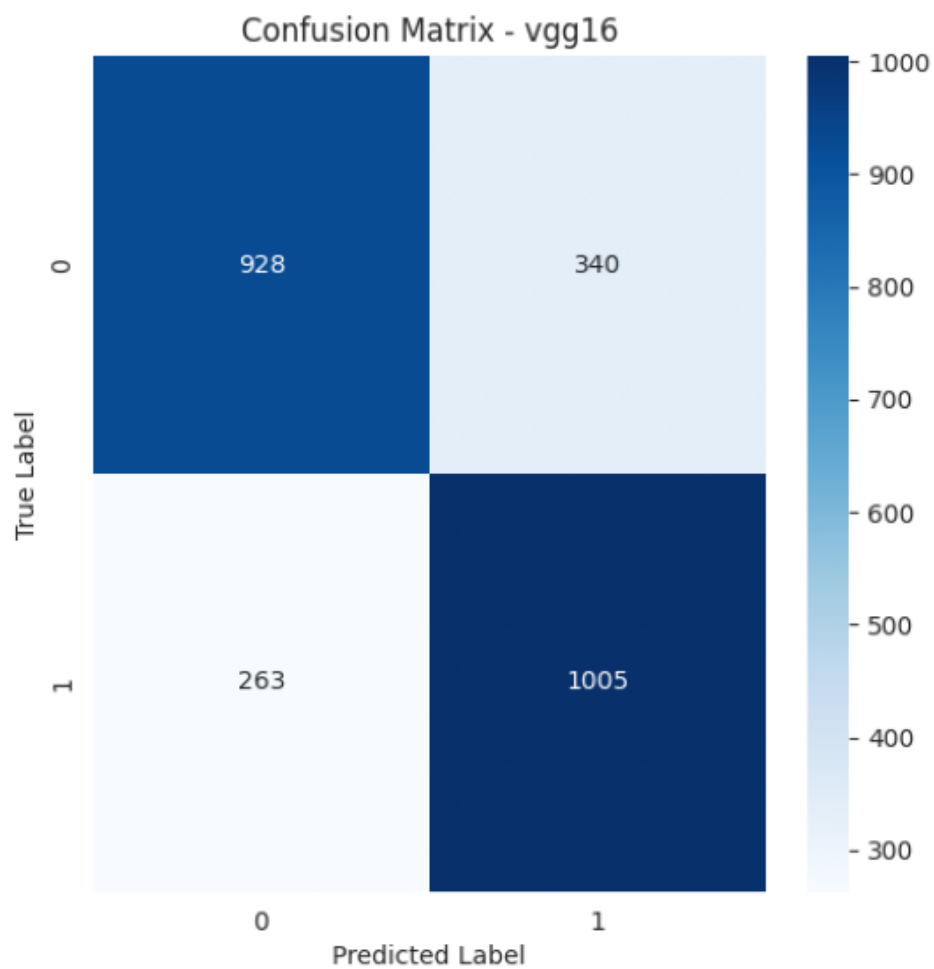
image_extensions = (".png", ".jpg", ".jpeg", ".bmp", ".gif", ".tiff")
image_files = [f for f in os.listdir(path) if
f.lower().endswith(image_extensions)]

for img_file in image_files:
    img_path = os.path.join(path, img_file)
    img = mpimg.imread(img_path)
    plt.figure(figsize=(8, 8))
    plt.imshow(img)
    plt.axis("off")
    plt.title(img_file)
    plt.show()
```

cm_inception.png



cm_vgg16.png



comparison_plot.png

